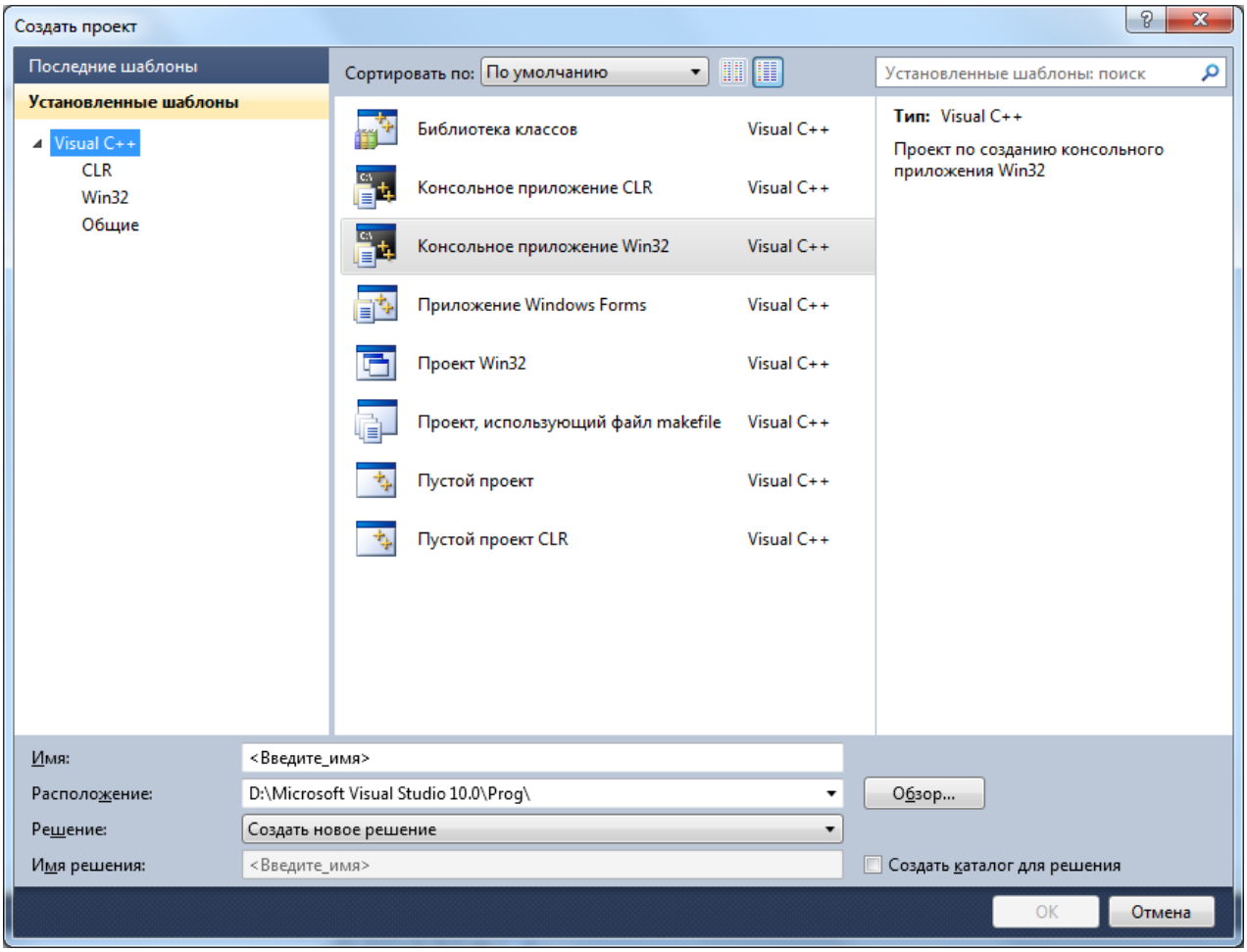
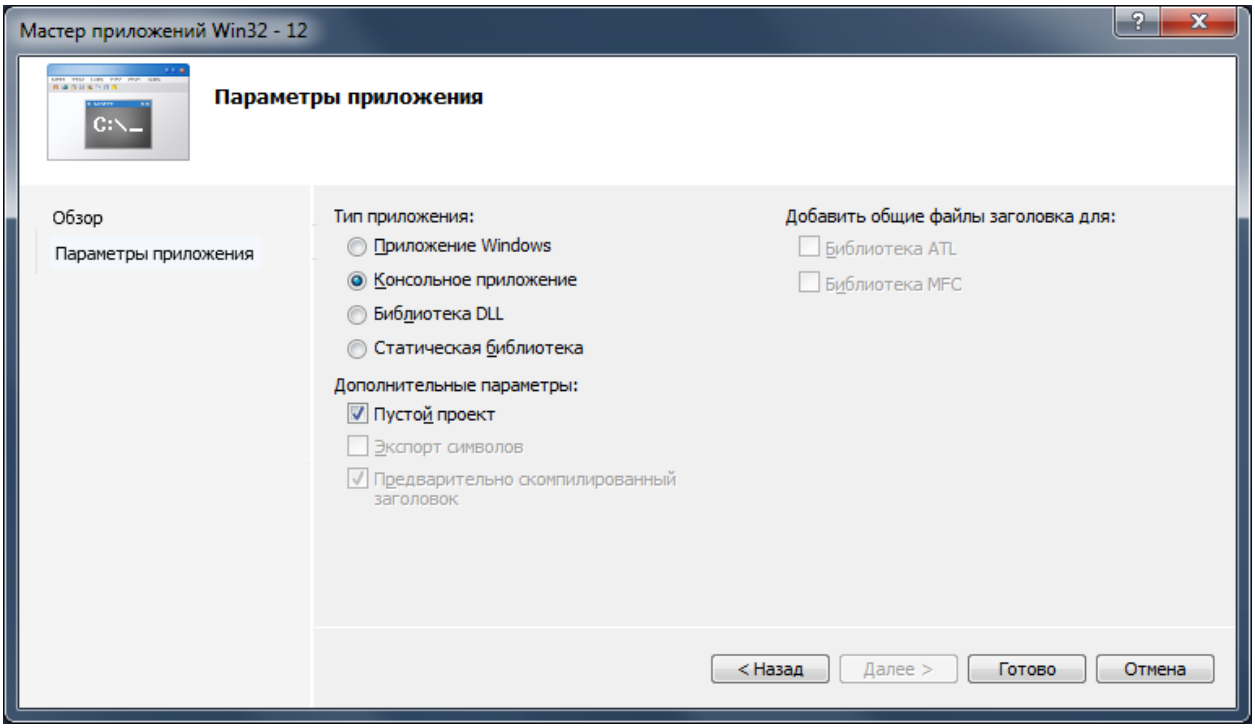
1. Создание проекта с поддержкой OpenMP в среде Microsoft Visual C++

1. Запустите Microsoft Visual C++.
2. Выберите
   File-New-Project…
   Появится окно создания проекта. Выберите тип проекта «*Win32*», шаблон – «*Win32 Console Application*». Введите имя проекта, выберите папку для хранения проекта, уберите галочку «*Create directory for solution*»:

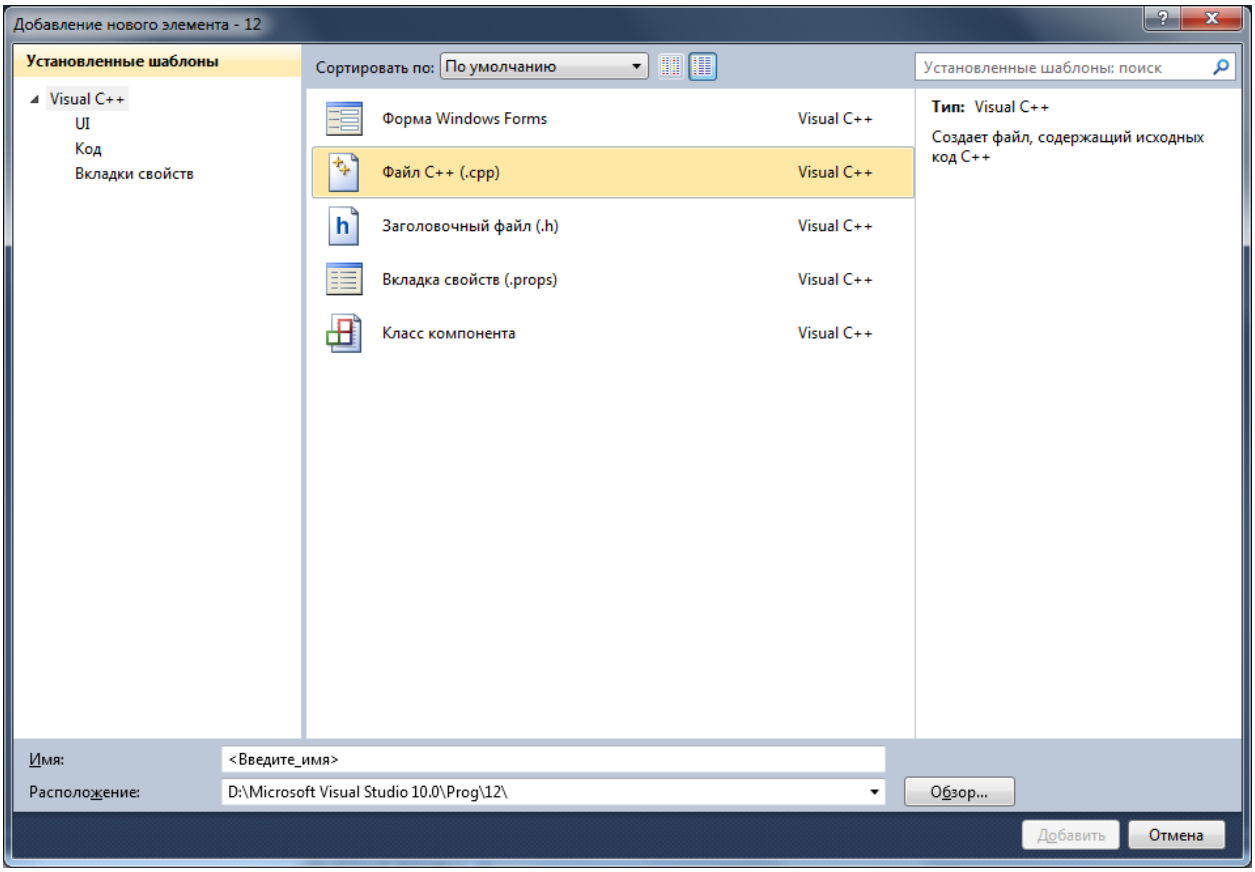


   Нажмите кнопку ОК, появится окно настройки будущего проекта. Выберите вкладку «*Application Settings*», и включите галочку «*Empty project*»:



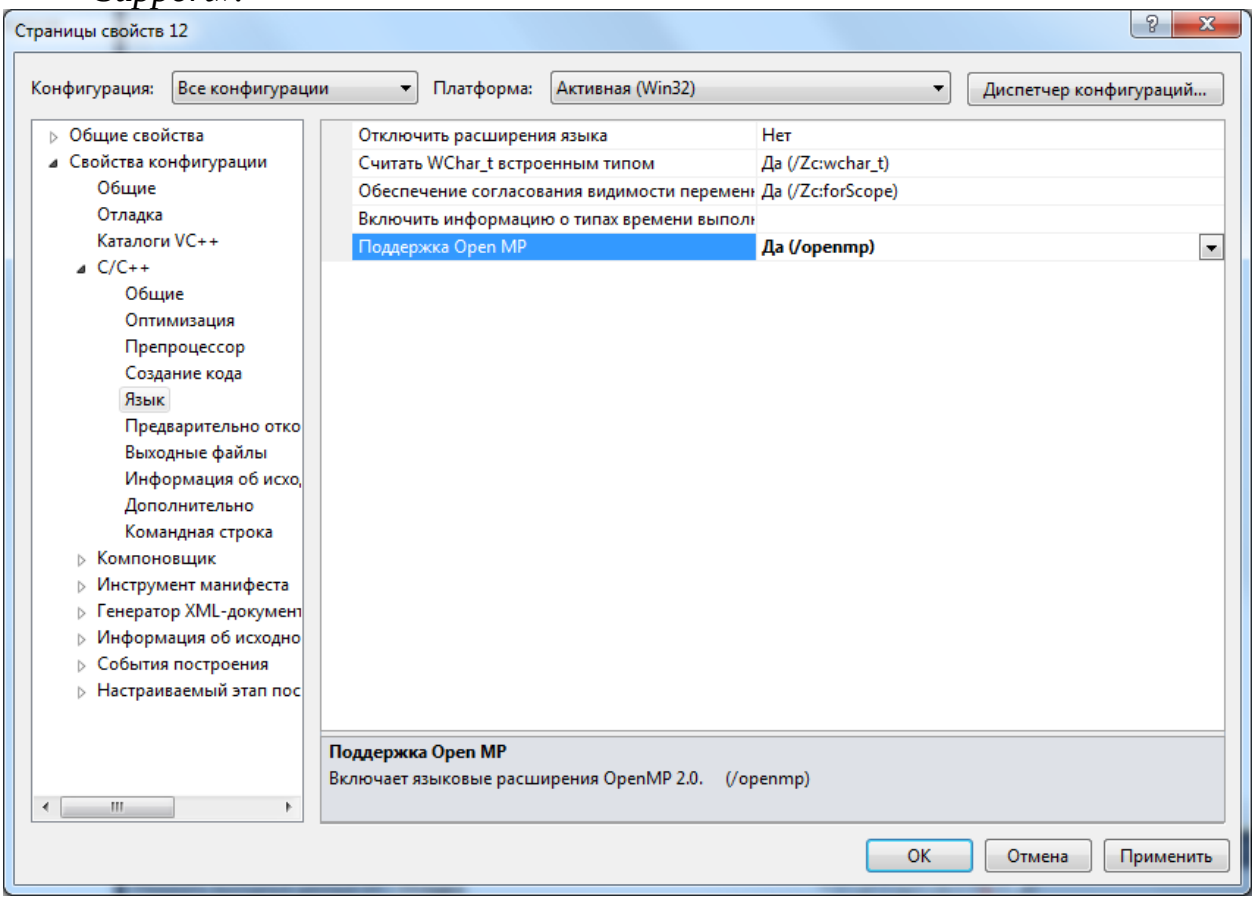
По нажатию кнопки «*Finish*» проект будет создан.

3. Нажмите *Project-Add New Item*, появится окно добавления элементов в проект. Добавьте «.cpp-файл» в проект:



После этого вам будет предоставлено окно для ввода исходного кода программы.

4. Для включения OpenMP нажмите *Project-Properties*. Слева вверху появившегося окна выберите «*All Configurations*» и в разделе *Configuration Properties-C/C++-Language* включите «*Open MP Support*»:



2. Структура OpenMP

Конструктивно в составе технологии OpenMP можно выделить:

• Директивы,
• Библиотеку функций,
• Набор переменных окружения.

Формат директив OpenMP

В самом общем виде формат директив OpenMP может быть представлен в следующем виде:

```
#pragma omp <имя_директивы> [<параметр>[[,]
<параметр>]…]
```

Начальная часть директивы (`#pragma omp`) является фиксированной, вид директивы определяется ее именем (имя_директивы), каждая директива может сопровождаться произвольным количеством параметров.

Для задания директивы может быть использовано несколько строк программы – признаком наличия продолжения является знак обратного слеша "\".

Действие директивы распространяется, как правило, на следующий в программе оператор, который может быть, в том числе, и структурированным блоком.

Директива *parallel* для определения параллельных фрагментов

Для выделения параллельных фрагментов программы следует использовать директиву `parallel`:

```
#pragma omp parallel [<параметр> ...]
<блок_программы>
```

Для блока (как и для блоков всех других директив OpenMP) должно выполняться правило «один вход – один выход», т.е. передача управления извне в блок и из блока за пределы блока не допускается.

Директива `parallel` является одной из основных директив OpenMP. Правила, определяющие действия директивы, состоят в следующем:

• Когда программа достигает директиву `parallel`, создается набор (*team*) из *N* потоков; исходный поток программы является основным потоком этого набора (*master thread*) и имеет номер 0.

• Программный код блока, следующий за директивой, дублируется или может быть разделен при помощи директив между потоками для параллельного выполнения.

• В конце программного блока директивы обеспечивается синхронизация потоков – выполняется ожидание окончания вычислений всех потоков; далее все потоки завершаются – дальнейшие вычисления продолжает выполнять только основной поток (в зависимости от среды реализации OpenMP потоки могут не завершаться, а приостанавливаться до начала следующего параллельного фрагмента – такой подход позволяет снизить затраты на создание и удаление потоков).

3. Пример первой параллельной программы

Приведем практически стандартную первую программу, разрабатываемую при освоении новых языков программирования – программу, осуществляющую вывод информации на экран.

Пример 1

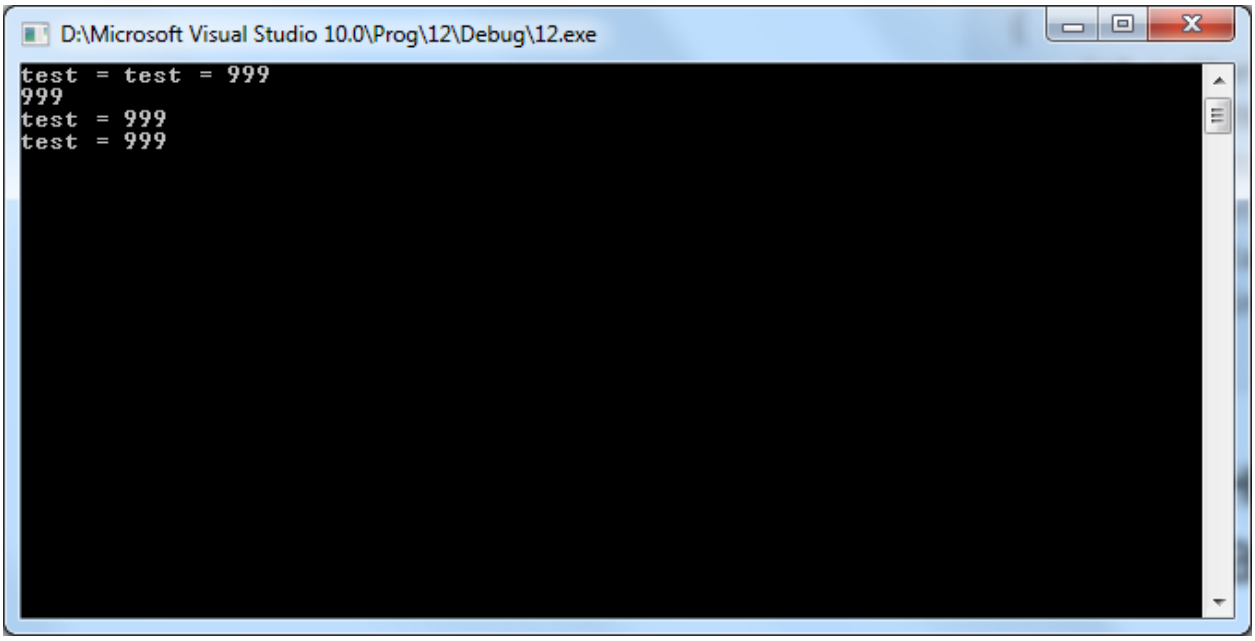
```
#include <iostream>
#include <omp.h>
#include <conio.h>
using namespace std;
void main(int argc, char **argv)
```

```
{
    int test=999;
  #pragma omp parallel
    {
       #pragma omp critical
        cout << "test = "<< test << endl;
    }
  getch();
}
```

В приведенной программе файл *omp.h* содержит определения именованных констант, прототипов функций и типов данных OpenMP – подключение этого файла является обязательным, если в программе используются функции OpenMP.

При выполнении программы по директиве `parallel` будут созданы потоки (по умолчанию их количество совпадает с числом имеющихся вычислительных элементов системы – процессоров или ядер), каждый поток выполнит программный блок, следуемый за директивой, и, как результат, программа выведет информацию на экран столько раз, сколько будет иметься потоков.

Использование директивы `critical` позволяет выполнять действие вывода информации на экран только одним из потоков в каждый момент времени ввиду того, что операция вывода строки на экран не является атомарной. Следовательно, если не использовать данную директиву, потоки будут выводить свои символы одновременно:



4. Определение времени выполнения параллельной программы

Практически сразу после разработки первой параллельной программы появляется необходимость определения времени выполнения вычислений, поскольку в большинстве случаев основной целью использования параллельных вычислительных систем является сокращений времени выполняемых расчетов. Используемые обычно средства для измерения времени работы программ зависят, как правило, от аппаратной платформы, операционной системы, алгоритмического языка и т.п.

Стандарт OpenMP включает определение специальных функций для измерения времени, использование которых позволяет устранить зависимость от среды выполнения параллельных программ.

Получение текущего момента времени выполнения программы обеспечивается при помощи функции:

```
double omp_get_wtime(void);
```

результат вызова которой есть количество секунд, прошедших от некоторого определенного момента времени в прошлом. Этот момент времени в прошлом, от которого происходит отсчет секунд, может зависеть от среды реализации OpenMP и, тем самым, для ухода от такой зависимости функцию `omp_get_wtime` следует использовать только для определения длительности выполнения тех или иных фрагментов кода параллельных программ. Возможная схема применения функции `omp_get_wtime` может состоять в следующем:

```
double t1, t2, dt;
t1 = omp_get_wtime ();
…
t2 = omp_get_wtime ();
dt = t2 - t1;
```

5. Управление количеством потоков

По умолчанию количество создаваемых потоков определяется реализацией и обычно совпадает с числом имеющихся вычислительных элементов в системе (процессоров и/или ядер). При необходимости количество создаваемых потоков может быть изменено – для этой цели в OpenMP предусмотрено несколько способов действий.

Прежде всего, количество создаваемых потоков может быть задано при помощи параметра *num_threads* директивы `parallel`.

Количество необходимых потоков может быть также задано при помощи функции `omp_set_num_threads` библиотеки OpenMP.

Формат функции `omp_set_num_threads` имеет вид:

```
void omp_set_num_threads (int num_threads);
```

Вызов функции omp_set_num_threads должен осуществляться из последовательной части программы.

Для задания необходимого количества потоков можно воспользоваться и переменной окружения OMP_NUM_THREADS. При использовании нескольких способов задания наибольший приоритет имеет параметр *num_threads* директивы parallel, затем функция библиотеки, затем переменная окружения.

Изменение количества создаваемых потоков может быть полезно и для целей отладки разрабатываемой параллельной программы для проверки ее работоспособности при разном количестве потоков.

Приведем здесь также дополнительные функции библиотеки OpenMP, которые могут быть использованы при работе с потоками:

• Получение максимально-возможного количества потоков:

```
int omp_get_max_threads(void)
```

• Получение фактического количества потоков в параллельной области программы:

```
int omp_get_num_threads(void)
```

• Получение номера потока:

```
int omp_get_thread_num(void)
```

• Получение числа вычислительных элементов (процессоров или ядер), доступных приложению:

```
int omp_get_num_procs(void)
```

Пример 2

```
void main(int argc, char **argv)
 {
 cout<<"Max num threads="<<omp_get_max_threads()<<endl;
 omp_set_num_threads(2);
#pragma omp parallel num_threads(3)
{
 cout<<"Parallel region 1 \n";
 cout<<"Thread num="<<omp_get_thread_num()<<endl;
}
#pragma omp parallel
{
 cout<<"Parallel region 2 \n";
 cout<<"Thread num="<<omp_get_thread_num()<<endl;
}
   getch();
 }
```

Литература

1. **Антонов А.С.** Введение в параллельные вычисления. – Москва: Изд-во МГУ, 2002. – 69 с.
2. **А. А. Букатов, В. Н. Дацюк, А. И. Жегуло**. Программирование многопроцессорных вычислительных систем. Ростов-на-Дону. Издательство ООО «ЦВВР», 2003, 208 с.
3. **Гергель В**.П. Параллельное программирование с использованием OpenMP. Нижний Новгород: Изд-во НГУ, 2007. – 33 с.